Demand Paging and Page Replacement

How a computer runs programs bigger than its RAM, and how it decides what to throw out

Operating Systems · Memory Management

Six questions we will answer

PART 1

Why not load everything?

Demand paging, locality

PART 2

Where do pages wait?

Swap space, PTE bits

PART 3

What is a page fault?

The handler, step by step

PART 4

What does it cost?

Effective access time

PART 5

Who gets evicted?

OPT, FIFO, LRU, Clock

PART 6

When does it collapse?

Thrashing, working set

Recap: pages, frames and the page table

PAGE

A fixed-size block of a process's virtual address space.

Typically 4 KB.

FRAME

A block of physical RAM, the same size as a page.

Any page can go in any frame.

PAGE TABLE

A per-process map from page number to frame number.

Written by the OS, read by the MMU on every access.

Worked translation (4 KB pages)

Virtual address	0x3A7F
Page number	3
Offset	0xA7F
Page table says	page 3 → frame 9
Physical address	**0x9A7F**

The offset is copied unchanged. Only the page number is translated.

Programs ask for far more memory than exists

256 TB

Address space of one 64-bit process on x86-64 (2^48 bytes)

8–16 GB

RAM in a typical laptop

Not necessary to load it all: a program uses only a small part of its memory at any moment.

Not possible to load it all: RAM is thousands of times too small.

So the OS hands out more memory than it has. This is called **overcommitting** memory.

Virtual memory and demand paging

VIRTUAL MEMORY

Giving each process the illusion of a large private memory, larger than physical RAM.

Some pages live in RAM, the rest on disk.

DEMAND PAGING

Load a page into RAM only when the process first accesses it.

Pages never touched are never loaded.

ANALOGY

A 300-page textbook.

Only the chapter you are reading is on your desk (RAM). The rest stays on the shelf (disk). You fetch a chapter when you need it.

Pure demand paging starts with zero pages in RAM. Real systems also **prefetch** nearby pages they expect to need soon.

Why it works: locality of reference

LOCALITY OF REFERENCE

Programs access a small, slowly changing set of pages at any time.

So after a few early loads, most accesses find their page already in RAM.

TEMPORAL LOCALITY

Data used now will likely be used again soon.

A loop counter, a function called in a loop.

SPATIAL LOCALITY

Data near what is used now will likely be used soon.

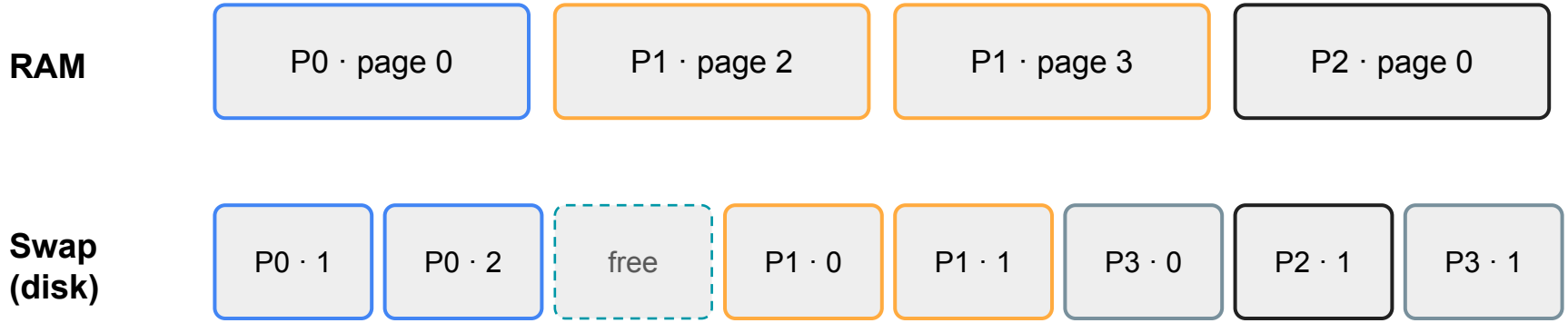
Walking through an array, running instructions in order.

Swap space

SWAP SPACE

A reserved area on disk that holds pages that do not fit in RAM.

Pages are written out when RAM is full and read back in when accessed.



P3 has no pages in RAM at all: it is fully swapped out.

What a page table entry records

Field	Meaning	Set by
Valid bit	The page is part of the process's address space (the program may use it).	OS
Present bit	The page is in RAM right now.	OS
Frame no. / disk address	Where the page is: a frame if present, a swap location if not.	OS
Dirty bit	Written since loaded, so the disk copy is out of date.	Hardware (MMU)
Accessed bit	Used recently. Read by replacement policies.	Hardware (MMU)

The present bit tells the OS how to read the address field: as a frame number or as a disk location.

Valid and present: the four cases

	Present = 1	Present = 0
Valid = 1	**In RAM** normal access	**In swap** page fault, load it
Valid = 0	**Impossible** never happens	**Not yours** segmentation fault

TEXTBOOK WARNING

Galvin uses one "valid" bit for both ideas.

In Galvin, "invalid" means either "not in RAM" or "not yours". The OS checks its own records to tell which. These slides split them into two bits.

Page fault

PAGE FAULT

A hardware trap raised when a process accesses a page that cannot be used directly, most often because it is not in RAM.

The CPU switches to kernel mode and runs the OS's page fault handler.

VALID, NOT PRESENT

Load the page

Normal and expected. The process continues.

NOT VALID

Terminate

Segmentation fault: the address is not the program's.

BAD PERMISSION

Terminate

For example, writing to a read-only page.

A page fault is not an error. Most faults are part of normal demand paging.

How the OS handles a page fault

1. MMU finds present = 0 and traps to the OS.
2. OS checks the access is legal. If not, terminate.
3. Find a free frame. If none, choose a victim page and evict it.
4. Read the page from disk. The process is blocked during I/O.
5. Update the page table: frame number, present = 1.
6. Restart the faulting instruction.

VICTIM PAGE

The page chosen to leave RAM to make room.

Choosing it well is the job of the replacement policy (part 5).

While the process waits for the disk, the scheduler runs another process. The CPU is not idle.

Minor and major page faults

MINOR FAULT

Handled without reading the disk.

The page is already in memory somewhere (for example a shared library), or it is a brand-new page that just needs a zero-filled frame. Takes microseconds.

MAJOR FAULT

Needs a disk read.

The page is in swap or in a file on disk. Takes milliseconds: about a thousand times slower.

Try it: on Linux, /usr/bin/time -v ls prints both counts as "Minor page faults" and "Major page faults".

Evicting a victim page

1. If the victim is dirty, write it to swap.
2. Mark its PTE present = 0, store its disk address.
3. Remove its stale entry from the TLB.
4. Load the new page into the freed frame.

LOCAL VS GLOBAL REPLACEMENT

Local: evict only from the faulting process. Global: from any process.

Global uses memory better; local stops processes hurting each other.

Real OSes keep a pool of free frames using a background thread (Linux kswapd), so most faults find a frame at once.

File-backed, anonymous and dirty pages

FILE-BACKED PAGE

Content comes from a file.

Program code, mapped files.

ANONYMOUS PAGE

No file behind it.

Heap and stack.

DIRTY PAGE

Changed since it was loaded.

Must be written back first.

Page type	Evicting when clean	Evicting when dirty
Program code	Just drop it (re-read from the executable)	Never dirty (read-only)
Mapped file	Just drop it	Write back to the file
Heap or stack	Drop if swap already has a copy	Write to swap

Effective access time

EFFECTIVE ACCESS TIME (EAT)

The average time for one memory access, counting the chance of a page fault.

$$\text{EAT} = (1 - p) \times \text{memory time} + p \times \text{fault time}$$

Memory access	200 ns
Fault service	8 ms = 8,000,000 ns
Fault rate p	1 in 1,000
EAT	**≈ 8,200 ns**

41× slower

One fault per thousand accesses. To stay within 10%, faults must be rarer than 1 in 400,000.

Three things can happen on a memory access

FASTEST

TLB hit

Translation found in the TLB. One memory access.

~100 ns

SLOWER

TLB miss, page in RAM

MMU walks the page table, fills the TLB, then accesses.

~200+ ns

SLOWEST

Page fault

Page not in RAM. Trap to OS, disk I/O, restart.

~ms

TLB (TRANSLATION LOOKASIDE BUFFER)

A small, fast cache of recent page-to-frame translations inside the CPU.

Page Replacement

PAGE REPLACEMENT POLICY

The rule the OS uses to choose a victim page.

Goal: fewest page faults.

REFERENCE STRING

The sequence of page numbers a process accesses.

Example: 0 1 2 0 1 3 0 3 1 2 1

HIT RATE

Hits ÷ total accesses.

COMPULSORY (COLD) MISS

The first access to a page. Unavoidable.

CAPACITY MISS

A miss because memory was too small to keep the page.

The only kind a policy can reduce.

OPT: evict the page needed furthest ahead

Step	Page	Result	Evict	Frames
1	**0**	Miss		0
2	**1**	Miss		0, 1
3	**2**	Miss		0, 1, 2
4	**0**	Hit		0, 1, 2
5	**1**	Hit		0, 1, 2
6	**3**	Miss	**2**	0, 1, 3
7	**0**	Hit		0, 1, 3
8	**3**	Hit		0, 1, 3
9	**1**	Hit		0, 1, 3
10	**2**	Miss	**0**	2, 1, 3
11	**1**	Hit		2, 1, 3

When 3 arrives, memory holds 0, 1, 2. Next uses: 0 at step 7, 1 at step 9, 2 at step 10. Evict 2.

CATCH

Needs to know the future.

Cannot be built. Used as the best-possible benchmark.

Evict the page in memory whose next use is furthest in the future. A page that is never used again is the best victim of all.

6 hits, 5 misses. Hit rate 6/11 = 54.5%.

FIFO: evict the page that arrived first

Step	Page	Result	Evict	Frames
1	**0**	Miss		0
2	**1**	Miss		0, 1
3	**2**	Miss		0, 1, 2
4	**0**	Hit		0, 1, 2
5	**1**	Hit		0, 1, 2
6	**3**	Miss	**0**	3, 1, 2
7	**0**	Miss	**1**	3, 0, 2
8	**3**	Hit		3, 0, 2
9	**1**	Miss	**2**	3, 0, 1
10	**2**	Miss	**3**	2, 0, 1
11	**1**	Hit		2, 0, 1

When 3 arrives, FIFO evicts 0, the oldest page. But 0 is the most used page, and it is needed again on the very next access.

WEAKNESS

Ignores how often a page is used.

Simple to build: just a queue.

4 hits, 7 misses. Hit rate 4/11 = 36.4%.

LRU: evict the least recently used page

Step	Page	Result	Evict	Frames
1	**0**	Miss		0
2	**1**	Miss		0, 1
3	**2**	Miss		0, 1, 2
4	**0**	Hit		0, 1, 2
5	**1**	Hit		0, 1, 2
6	**3**	Miss	**2**	0, 1, 3
7	**0**	Hit		0, 1, 3
8	**3**	Hit		0, 1, 3
9	**1**	Hit		0, 1, 3
10	**2**	Miss	**0**	2, 1, 3
11	**1**	Hit		2, 1, 3

Uses the past to predict the future. Because of locality, a page not used for a while is unlikely to be used soon.

RESULT

Matches OPT on this string.

A coincidence of this example, not a rule.

6 hits, 5 misses. Hit rate 6/11 = 54.5%.

Same strings, three policies

Reference string (3 frames)	OPT	LRU	FIFO
0 1 2 0 1 3 0 3 1 2 1	5	5	7
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1	9	12	15
1 2 3 4 1 2 5 1 2 3 4 5	7	10	9

- Numbers are page faults. Lower is better.
- OPT is always lowest: it is the floor. LRU usually comes close.
- FIFO is usually worst, but not always: on the last string LRU loses to FIFO. No practical policy wins on every string.

Belady's anomaly: more frames, more faults

BELADY'S ANOMALY

Giving FIFO more frames can increase the number of page faults.

Frames	1	2	3	4	1	2	5	1	2	3	4	5	Faults
3 frames	F	F	F	F	F	F	F	H	H	F	F	H	9
4 frames	F	F	F	F	H	H	F	F	F	F	F	F	10

WHY LRU AND OPT ARE SAFE

They are stack algorithms.

Pages kept with n frames are always a subset of those kept with n + 1, so an extra frame can only help.

Exact LRU is too expensive

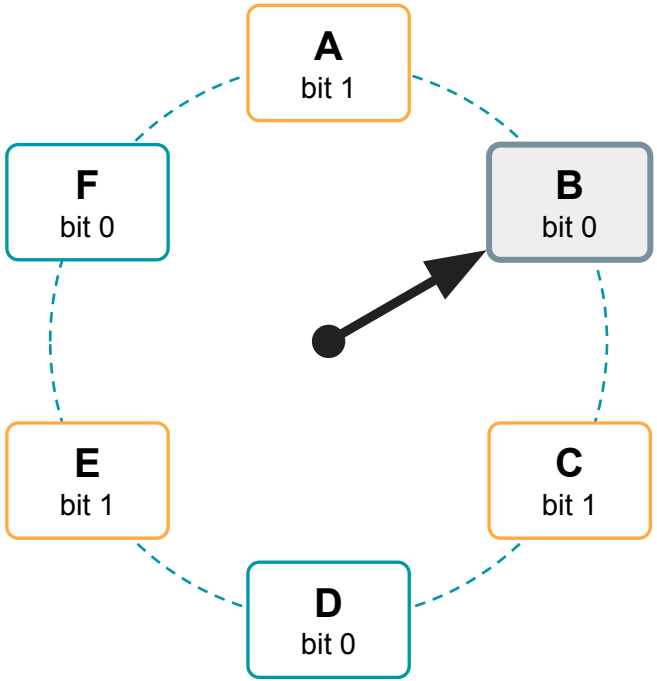
- Exact LRU must update a timestamp or list on every memory access.
- That is billions of updates per second. Hardware cannot afford it, and the OS does not even see normal accesses.
- Instead, the MMU sets one accessed bit per page. The OS reads and clears these bits to approximate LRU.

ACCESSED (REFERENCE) BIT

A PTE bit that hardware sets to 1 whenever the page is used.

1 means "used since the OS last cleared it".
Cheap: one bit, set by hardware.

Clock (second chance): approximate LRU



1. On a fault, look at the page under the hand.
2. Bit = 1: set it to 0 (a second chance), move on.
3. Bit = 0: evict it, put the new page here with bit 1.

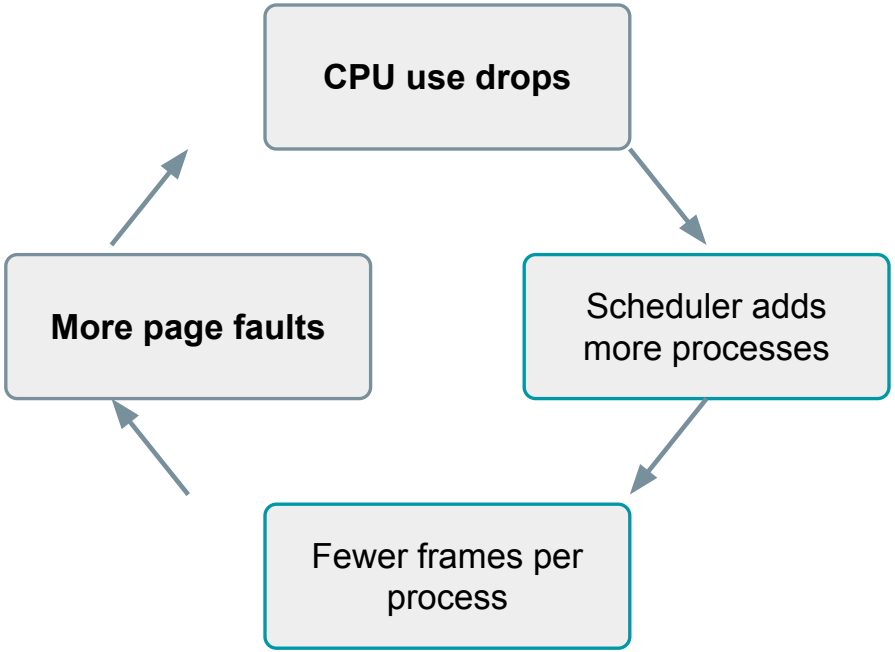
In the picture the hand passed A (bit 1 → 0) and stops at B (bit 0): **B is the victim.**

Thrashing

THRASHING

The system spends more time moving pages to and from disk than doing useful work.

Cause: processes have fewer frames than the pages they actively use.



No replacement policy can fix this. Every page evicted is needed again almost at once.

Working set and page-fault frequency

WORKING SET WS(T, Δ)

The distinct pages a process used in its last Δ references.

The pages it needs in RAM now to run without thrashing.

Last 10 references	Working set
2 6 1 5 7 7 7 7 5 1	{1, 2, 5, 6, 7}
3 4 4 4 3 4 3 4 4 4	{3, 4}

Δ = 10. The working set changes as the program moves between phases.

WORKING-SET RULE

If all working sets together exceed RAM, suspend a process.

Swap it out whole until the rest fit.

PAGE-FAULT FREQUENCY (PFF)

Watch each process's fault rate.

Too high: give it more frames. Too low: take some away. None free: suspend a process.

Summary: the key terms

Demand paging	Load a page only when it is first accessed.
Swap space	Disk area holding pages that do not fit in RAM.
Page fault	Trap when an accessed page cannot be used directly, usually because it is not in RAM.
Victim page	The page evicted to make room. Dirty victims are written to disk first.
Effective access time	Average access time including faults. One fault per 1,000 accesses ≈ 41× slower.
OPT · FIFO · LRU · Clock	Evict: furthest future use · oldest · least recently used · first page with bit 0.
Belady's anomaly	FIFO can fault more with more frames. LRU and OPT cannot.
Thrashing	More time paging than working. Fixed by working-set or PFF control.

Check yourself

1. A process reads a page with valid = 1, present = 0. List what the OS does, in order.

2. Trace 2 3 2 1 5 2 4 5 3 2 5 2 with 3 frames. Count faults for FIFO, LRU and OPT.

3. Memory takes 100 ns and a fault takes 10 ms. What fault rate keeps EAT under 200 ns?